Plan de corrección del informe «Robótica educativa y estadística»

Estado al 20/10/2026. Documento base: 17 páginas, «Informe Robótica Educativa.pdf». Objetivo: cumplir las pautas del IEN y describir únicamente hechos y resultados demostrables.

Hechos comprobados por el grupo

- Armado iniciado el 16/10/2026 y finalizado el 20/10/2026.
- Prueba de desplazamiento con la aplicación oficial ACEBOTT por Wi-Fi el 20/10/2026: respuesta correcta al control manual.
- Sensor ultrasónico instalado. NO hay todavía algoritmo autónomo implementado ni experimentos estadísticos ejecutados.
- Se proyecta utilizar MicroPython en el ESP32, sujeto a validar el mapa de pines y resguardar el firmware existente.

Condiciones del IEN

- Entrega: viernes 6/11/2026; defensa: viernes 13/11/2026.
- A4; márgenes 2,5 cm; Arial/Times New Roman 12 pt; títulos 14 pt negrita; texto justificado; interlineado 1,5; páginas de desarrollo numeradas.
- 8–15 páginas de desarrollo (no cuentan portada, índice ni anexos), salvo indicación de docente; entrega digital e impresa.
- Los títulos 1–10 se mantienen en el orden solicitado.
- La rúbrica de 100 puntos evalúa la defensa oral; 20 puntos de estadística y 15 de programación/robótica. No confundir con calificación del informe escrito.

Correcciones por página y sección

Página	Sección	Acción puntual
1	Portada	Cambiar fecha de entrega de 30/10 a 06/11; revisar tildes de nombres con los integrantes. Mantener título por ahora como proyecto previsto; confirmar «programado en Python» antes de entrega.

Página	Sección	Acción puntual
2	Índice	Reconstruir una vez finalizada la paginación.
3	Introducción	Sustituir las afirmaciones en pasado «datos obtenidos», «su programa» y «conclusiones» por descripción neutra de estructura, hasta tener datos. Incorporar autonomía y análisis de frenado como foco.
4	Objetivos	En general, «desarrollar un algoritmo autónomo con detección de obstáculos y evaluar su desempeño». Ajustar específicos a lo efectivamente probado; no declarar aún cumplidos.
5	Marco 5.2	Quitar sección extensa Arduino UNO/pyFirmata y enlace de foro; reemplazar referencia de RP2 (Pico) por manual MicroPython ESP32. Describir `machine.Pin`, PWM, medición de pulso, USB/serial. Aclarar diferencia Python en PC frente a MicroPython nativo.
6	5.3 sensores	Sustituir como referencia principal las cifras del HC-SR04 genérico por la ficha ACEBOTT QB042/QB093 **solo si se confirma modelo instalado**. Cifras de catálogo (3–500 cm, 0,1 cm declarados, 40 kHz); aclarar que la exactitud real se evaluará. No asegurar driver L298 sin identificador verificable; retirar página ST si no aplica.
6–7	5.4–5.7	Mantener error, media, mediana, s, binomial, IC y regresión; revisar tipografía de fórmulas, hipótesis de independencia y normalidad. Aclarar correlación alta ≠ precisión/ausencia de sesgo; usar residuos y error absoluto.

Página	Sección	Acción puntual
8	5.8 antecedentes	Mantener Zapata y Hunter como antecedentes de sensores ultrasónicos de bajo costo, indicando que se realizaron sobre HC-SR04 y pueden NO ser el mismo modelo del QD001.
9	6.1 prototipo	Confirmar tareas: desplazamiento autónomo, detección y frenado. Sustituir nota `[COMPLETAR]` por propósito previsto, sin afirmar autonomía lograda. Agregar foto propia cuando esté.
9	6.2 componentes	Enumerar lo efectivamente instalado; tabla con modelo, función, cantidad, fuente y estado de identificación. No inventar pines/driver.
10	6.3 armado	Reemplazar `[COMPLETAR]` con el texto de armado 16–20/10 y prueba manual Wi-Fi. Agregar dificultades reales solo si ocurrieron. Diagramar conexiones conforme a fuente específica Car Shield V2.0.
10	6.4 código	Declarar MicroPython como lenguaje elegido, todavía no implementado; describir ciclo leer–validar–decidir–actuar–registrar como diseño, no logro. Reemplazar por código real y diagrama cuando Codex lo valide.
10–12	6.5 pruebas	Elegir tres ensayos prioritarios (repetibilidad, calibración, frenado). PWM opcional. Redactar protocolo con unidad de observación, referencia, n, superficie, batería, objeto, condiciones, criterio de éxito, datos a registrar y manejo de fallos.
12	6.6 seguridad	Desconectar antes de tocar cables; limitar potencia, elevar ruedas primer encendido, botón/acción de parada, validar ECHO 3,3 V, detener ante lectura inválida. Diferenciar riesgos conocidos de hallazgos reales.

Página	Sección	Acción puntual
13	7 resultados	No completar con datos hipotéticos. Reservar tablas de n, media, mediana, s, IC; regresión/MAE; éxitos e intentos; insertar valores reales solamente tras ensayos.
14	8 conclusión	No escribir resultados numéricos ni afirmar autonomía lograda antes de los ensayos. Redactar al final contra cada objetivo.
15–16	9 bibliografía	Mantener fuentes comprobadas y citadas; sustituir enlaces inaccesibles o fuera de tema; registrar fecha real de consulta. Ver auditoría de enlaces.
17	10 anexos	A: código real comentado; B: CSV sin procesar; C: fotos de armado y experimentos; D: documentación técnica real; E: scripts/cálculos/gráficos.

Textos propuestos para reemplazar (secciones con hechos conocidos)

6.3 Experiencia de armado — párrafo listo

> El armado del robot educativo ACEBOTT QD001 se inició el 16 de octubre de 2026 y finalizó el 20 de octubre del mismo año. Con el prototipo ensamblado, el grupo realizó una comprobación inicial del sistema de desplazamiento mediante la aplicación oficial de ACEBOTT, conectada al robot a través de Wi-Fi. El vehículo respondió a las órdenes de movimiento enviadas desde la aplicación, por lo que se verificó el funcionamiento básico de la tracción y del control manual. Esta prueba no permitió evaluar todavía el comportamiento autónomo ni la precisión del sensor ultrasónico. Como etapa siguiente, se proyectó implementar el control automático con MicroPython en el ESP32 y realizar ensayos repetidos de detección y detención ante obstáculos. [Agregar después una foto de la unidad y las dificultades de armado realmente observadas.]

5.2 MicroPython para ESP32 — sustitución sugerida

> MicroPython es una implementación de Python orientada a microcontroladores. En este proyecto se prevé utilizarlo en el ESP32 del ACEBOTT QD001 para ejecutar un algoritmo de desplazamiento au-

> tónomo sin necesidad de mantener conectada una computadora. La biblioteca `machine` permite acceder a entradas y salidas digitales, a señales PWM y a funciones temporales que pueden utilizarse para controlar actuadores y medir el eco de un sensor ultrasónico. El programa se diseñará en etapas: adquisición de la lectura, validación, toma de decisión, actuación y registro. El mapa concreto de conexiones y el driver de los motores deberán identificarse en el material técnico del Car Shield V2.0 antes de implementar el control. MicroPython puede reemplazar el firmware que habilita la aplicación oficial, motivo por el que se prevé respaldar primero la memoria del ESP32. [Actualizar al pasado únicamente cuando se haya ejecutado y comprobado el programa.]

6.1 Descripción de la tarea — incorporación sugerida

> La tarea propuesta es que el robot se desplace automáticamente y, al detectar un obstáculo dentro de un umbral de seguridad previamente definido, se detenga para evitar la colisión. Se medirá la repetibilidad de las distancias informadas por el sensor ultrasónico, se compararán las lecturas con distancias de referencia y se evaluará la proporción de frenadas correctas en ensayos repetidos. El éxito dependerá de un criterio definido antes de las pruebas y de condiciones de medición controladas. [No atribuir resultados a esta función hasta implementarla y comprobarla.]

6.4 Diseño del programa — redacción temporal

> El programa autónomo se encuentra en etapa de planificación. Se propone que el ESP32 lea periódicamente la distancia del sensor ultrasónico, descarte lecturas inválidas, compare el resultado con una distancia de seguridad y controle el movimiento mediante la etapa de potencia del robot. Ante un obstáculo próximo o una medición inválida, deberá ordenar la detención. Asimismo, registrará los datos necesarios para analizar experimentalmente el desempeño. La versión definitiva del algoritmo, los pines y los fragmentos de código se incorporarán después de las pruebas de programación. [Reemplazar la descripción de «se propone» por implementación real después de la validación.]

Metodología que falta cerrar antes de medir

Repetibilidad: tres distancias de referencia distintas y una cantidad fija de lecturas por distancia (se propone 30 como meta a consensuar). Medir desde un punto de referencia físico definido y no mover el objeto entre lecturas.

Calibración: 8–10 distancias reales bien separadas, varias lecturas en cada una. Registrar distancia real y medida, sesgo, error absoluto medio (MAE), recta de calibración, `r` y residuos.

Frenado: mismo obstáculo, batería y superficie, velocidad conocida, punto de inicio fijo y umbral de seguridad documentado. Definir «éxito» antes de ejecutar corridas (p. ej. frenar sin tocar el obstáculo y dentro de una banda de distancia definida por el docente y el protocolo). Contar éxitos/total; jamás elegir el criterio mirando los resultados.

PWM (opcional): tiempos para desplazarse una distancia fija a varios niveles de potencia, solo si se controla el hardware y queda tiempo.

Auditoría de bibliografía y enlaces (20/10/2026)

Accesibles y con contenido consultado/relevante: - NAP Educación Digital, Programación y Robótica, CFE 343/18. - Albarello y Hafner Táboas, Contratexto. Sustituir como enlace principal al DOI que el verificador no pudo abrir directamente. - NIST/SEMATECH, Measurement Process Characterization. - JCGM 100:2008 en BIPM. - MicroPython ESP32 Quick Reference. Sustituye RP2. - MicroPython `machine`. - ACEBOTT QD001. - ACEBOTT placa ESP32 Max V1.0. - ACEBOTT sensor QB042/QB093; solo atribuir datos al robot cuando se confirme módulo instalado. - ACEBOTT tutorial para Car Shield V2.0. - Zapata, arXiv:2506.06314. - ST L298: enlace accesible, **quitar del trabajo salvo identificación real de ese driver**.

No accesibles directamente desde el verificador o de pertinencia insuficiente: - DOI de Contratexto `https://doi.org/10.26439/contratexto2019.n032.4607`: el destino editorial sí está comprobado; utilizar ese destino. - DOI de Hunter `https://doi.org/10.62962/001c.87939`: acceso directo dio error; la referencia consta en la revista académica, con ficha indexada en https://ewu-urj.scholasticahq.com/article/87939-analysis-of-the-measurement-error-from-a-low-cost-ultrasonic-sensor ; acceso al texto no fue consistente. Mantener con nota de verificación pendiente o utilizar fuente accesible al cerrar. - `https://www-users.cs.york.ac.uk/~mjf/roach/Datasheets/HCSR04_Manual.pdf`: no accesible al verificador; retirar si el sensor no es HC-SR04. - `https://www.luisllamas.es/en/micropython-servomotors-pwm/`: respuesta HTTP 403; retirar si el servo no se analiza o reemplazar por fuente oficial accesible. - `https://docs.micropython.org/en/latest/rp2/quickref.html`: funciona, pero corresponde a **RP2/Raspberry Pi Pico, no ESP32**; reemplazar. - Foro Arduino UNO/pyFirmata: funciona, pero no justifica el diseño propio del QD001; eliminar de la versión final si se retira el apartado de UNO.

Norma de trabajo: enlace accesible ≠ fuente científicamente pertinente. No afirmar que se leyó o verificó contenido bloqueado. Toda figura, tabla, especificación o cifra técnica debe tener fuente asociada.

Pendientes que solo el grupo puede confirmar

- Si la placa instalada es exactamente la indicada en el PDF y si el Shield corresponde a V2.0.
- Modelo del sensor ultrasónico y número de GPIO realmente utilizados.
- Fotos del robot/armado/procedimiento; incidentes o dificultades reales del armado.
- Programa autónomo efectivamente ejecutado, versión, parámetros y pruebas.
- CSV reales, método de medida, criterios de éxito, duración y observaciones.
- Libro/manual de Probabilidad y Estadística utilizado por la cátedra, si corresponde.

Siguiente actualización del PDF

Primero editar 5.2, 5.3, 6.1–6.4 y bibliografía **sin fingir implementación**. Luego completar 6.5, 7, 8 y anexos con pruebas reales. Revisar formato académico, ecuaciones, títulos de tablas, índice, extensión e hipervínculos en el PDF exportado.